

TrustBank CBS — Architecture Discussion Brief

Internal briefing note · 21 August 2026 · For a management review of the .NET 10 migration

One page of "where we are", one of "what's open", and the decisions we need. Detail lives in the reference documents listed at the end.

1. The position in five lines

- We are migrating TrustBank CBS from **ASP.NET WebForms / VB.NET / .NET 4.8** to **.NET 10 / C# ASP.NET Core MVC**, using a **strangler-fig** approach — legacy keeps running, screens cut over one at a time.
 - The **reusable foundation is finished and proven**: framework, security, session, multi-database data layer, UI component toolkit, and a repeatable per-screen recipe. This is typically 15–20% of a programme of this size and it is behind us.
 - The target architecture — a **modular monolith** — is not a plan, it is **built**. As of **29 July 2026 all 8 business domains are their own assemblies** and the old shared service project was dissolved.
 - The platform runs today in **four deployment shapes** (IIS single, IIS multi-host behind a gateway, Docker single, Docker multi-host) from **one codebase**, and on **three databases** (SQL Server, Oracle, PostgreSQL).
 - An independent pre-production review scored the solution **62/100** in July; remediation has moved it to **~77**. **Two gate items remain**, and one of them — no source control or CI — is the single biggest thing to fix, and it now also gates the value of **four test suites — 380 automated tests** — that nothing runs automatically.
-

2. What is built

36 projects in the build solution, in four clean tiers.

Tier	Projects	What it gives us
Foundation	<code>Tf1Cbs.Abstractions</code> , <code>Tf1Cbs.Framework</code>	<code>Result<T></code> contract, module contract, the four framework ports, security guard, session, tokens, search infrastructure. Framework references <i>only</i> Abstractions — never a business service or a screen.
Business modules (8/8 promoted)	Core, always on: <code>General</code> , <code>Reference</code> , <code>Core.Authentication</code> · Domains: <code>Accounts</code> , <code>Administration</code> , <code>Clearing</code> , <code>HR</code> , <code>Lockers</code> , <code>RetailBanking</code>	Each domain is independently owned, separately buildable, and can be switched on or off by configuration.
Screens	5 domain web libraries (<code>*.Web</code>) + 2 core (auth, shell) + <code>Tf1Cbs.Web.Shared</code>	Screens ship as Razor Class Libraries, so a host composes exactly the screens it needs.
Hosts & infrastructure	<code>Tf1Cbs.Host.Main</code> (full host), YARP <code>Gateway</code> , 4 thin per- module hosts · <code>Tf1Cbs.Entities</code> , <code>Tf1OmniDb</code> , <code>Tf1OmniLog</code> , <code>Tf1SecurityCrypto</code> · <code>DbMigrator</code>	Same code, many footprints. In-house entity, data-access, logging and crypto libraries — no niche third-party dependencies. Schema changes are versioned and journalled by a migration runner.

Delivered functionality so far: 12 domain screens across 5 areas, plus login/logout, module chooser, menu, home and error handling — 47 views. These are the pilot screens the delivery estimate is calibrated on, not throwaway demos: they run against the real database.

Two things changed in August that are worth a line each. The two in-house libraries (`Tf10mniDb` , `Tf1SecurityCrypto`) were **vendored into the repository**, so the solution is now self-contained — previously a fresh copy could not be built unless an unrelated archive folder happened to sit at the right relative path, which is exactly the kind of thing that fails on the first day of a new team member or a new build agent. And the shared configuration files that are security controls (log masking) or operational controls (route cut-over) were moved out of the default host into a solution-level `config/` folder linked by all five hosts, so they cannot drift apart between hosts.

3. The architectural decisions, and why they were made

Modular monolith — one process, one database, hard internal boundaries.

This is the deliberate middle path between the legacy monolith and microservices, and for a core banking system it is the defensible choice:

- **Money movement needs transactions.** One database gives us real ACID guarantees across a posting. A distributed design would trade that for eventual consistency and a class of failure modes we would then have to engineer around — for no benefit at our volumes.
- **Operationally simple today.** One application to deploy, monitor and back up.
- **But the boundaries are real.** Modules talk to each other only through published contracts, and **94 automated architecture tests fail the build** if anyone crosses a line. This is the crucial difference from a conventional monolith: the design cannot silently erode as the team grows. The suite also guards *itself*: a meta-check verifies that each boundary rule is still genuinely capable of failing, so a rule cannot quietly turn green by becoming impossible to violate.
- **Scale-out is available without a rewrite.** Because the boundaries are enforced, we already run individual modules as separate hosts behind a gateway. That capability is built and demonstrated — we simply do not need it yet.

Second decision: horizontal framework/screen split. Reusable plumbing lives in one assembly that has no knowledge of any screen. That is what makes the per-screen migration recipe fast and repeatable, and it is likewise arch-tested.

4. What is proven, with evidence

Claim	Evidence
Boundaries hold	94 architecture tests , run on every build
Business logic works on all three databases	149 cross-provider service tests
The whole stack works end to end	74 in-process integration tests do a full authenticated round trip against the real pipeline — login, 22 module tiles, live menu, real search results from the database
The browser sees what we think it sees	63 Playwright end-to-end tests across 10 specs — the strict content-security policy actually enforced, session/cookie behaviour, and every reusable component driven for real
Access control is fail-closed	Central guard on every request + a startup assertion that it cannot be ordered away, both pinned by tests
No SQL injection surface	All access parameterized; new master CRUD is forbidden raw SQL by standing rule
Rewrites are no slower than the procs they replace	Dated performance reports comparing migrated C# against the original T-SQL, on both SQL Server and Oracle
The migration recipe is real	Documented step-by-step tutorial plus a written definition-of-done; a new developer builds a full CRUD screen from it

5. Scope and effort — the commercial picture

From an automated scan of the legacy codebase plus measured velocity on the delivered pilot screens (ROM, $\pm 40\%$, prepared 17 June 2026):

Legacy scope counted	1,458 ASPX screens (1,059 functional + 399 reports) · 5,317 stored procedures · 1,497 VB code-behind files
Likely effort	~10,550 person-days (~48 person-years)
Indicative timeline	~2.5–3 years with a team of ~20
Range	Optimistic ~7,400 pd · Likely ~10,550 pd · Pessimistic ~14,800 pd

The three biggest levers on that number: whether reports are rebuilt or rehosted ($\pm 500\text{--}700$ pd), whether we commit to one database or keep all three ($\pm 120\text{--}150$ pd), and how much senior attention the high-risk engine procedures (interest, GL, day-end, clearing) need.

6. Open risks — what we would want to say before anyone asks

#	Item	Status	Why it matters
1	No source control or CI	Open — highest priority, unchanged since July	The working tree is still not a git repository. Nothing is recoverable, nothing gates a bad change, and we have already kept dead code purely because we could not retrieve it later. Every other item on this list gets cheaper the moment this is fixed, because a fix can then be proven and kept. Cheap to do; it should not survive another sprint.

#	Item	Status	Why it matters
2	Optimistic concurrency not rolled out	Partial	The pattern is built and DB-verified on one master. Until it is applied to the remaining write services, concurrent edits are last-writer-wins.
3	Database credentials and certificates	Partial	Secrets are out of committed config, but the shared administrative login still needs rotating and replacing with a least-privilege account, and the self-signed certificate needs replacing for production.
4	No infrastructure-as-code; observability instrumented but disabled by default	Deferred by decision	These are the "next band" items after the production gate, not blockers. The controller and end-to-end test tiers that sat here in July are now done — 74 in-process integration tests and 63 browser tests.

Items 1–3 are the remainder of a written eight-item gate plan; six are already complete, including the two Critical findings (an authentication bypass and reversible password storage — passwords are now PBKDF2, with all 1,731 user records migrated).

The honest summary: the *structure* of this system is its strongest attribute and was rated as such independently. The remaining risk is almost entirely **process and operations**, not design.

7. What we would like decided

1. **Authorize source control and a CI pipeline now.** Small effort, removes the largest single risk, and every subsequent fix then lands behind a test gate.

2. **Settle the reports question** — rebuild or rehost. It is the biggest single swing in the estimate and it blocks a firm Phase-1 budget.
3. **Settle the database target** — commit to one engine, or fund keeping all three. We support three today; that has an ongoing hardening cost.
4. **Approve a 4–6 week discovery phase** to convert the $\pm 40\%$ ROM into a committed $\pm 15\%$ plan before any large build commitment. The foundation is ready, so the team can start migrating screens the day it ends.

Appendix — reference documents

Document	Audience	Formats
<code>architecture-overview.md</code>	The full engineering reference — every project, why it exists, and how a request flows end to end	md · html · txt · pdf · docx
<code>architecture/TrustBank-CBS-Platform-Architecture</code>	Outward-facing; for an institution evaluating the platform	md · html · txt · pdf · docx
<code>architecture-review/ARCHITECTURE-REVIEW</code>	The independent pre-production review, findings graded C/H/M	md · html · txt · pdf · docx
<code>architecture-review/REMEDIATION-PLAN</code>	The eight-item gate plan; live status	md · html · txt · pdf · docx
<code>architecture/modular-monolith.md</code>	Decision record — the vertical module split	md · html · txt · pdf · docx
<code>architecture/reusable-boundary.md</code>	Decision record — the framework/screen split	md · html · txt · pdf · docx

Document	Audience	Formats
migration-estimate/TrustBank-Migration-Estimate	Commercial pack — report, deck and workbook	docx · pptx · xlsx
port-map.md	Every host, port and deployment shape on one page	md · html
guides/day-one.md · guides/building-a-crud-screen.md · guides/screen-definition-of-done.md	The onboarding path: first-day setup, the per-screen recipe, and the review gate	md · html

Prepared from the solution as it stands on 21 August 2026. Figures for tests, projects and screens are counted from the source tree, not estimated.